PARIS CITE UNIVERSITY

RESEARCH PROJECT

Self-Supervised Test-Time Training for Image Classification

MAY 2026

Authors:

Rayane KHATIM
rayane.khatim@etu.u-paris.fr
22500285

Mehdi AGHAEI
mehdi.aghaei@etu.u-paris.fr
22500371

Supervisors:

Prof. Ayoub Karine : ayoub.karine@u-paris.fr
Prof. Camille Kurtz : camille.kurtz@u-paris.fr
Prof. Laurent Wendling : Laurent.Wendling@u-paris.fr

Contents

1 Introduction and Context

Image classification models usually work well when the test images are close to the images used during training. This assumption is weak in practice. A camera image can contain sensor noise, blur, compression artifacts, fog, snow, low contrast, or a strong change in brightness. These changes do not necessarily change the object class, but they change the input distribution seen by the model. A network trained only on clean images can then make unstable predictions, even when the image is still understandable for a human observer.

This project studies that problem in a controlled computer vision setting. CIFAR-10 is used as the clean source domain [6]. The target domain is based on corrupted CIFAR-10 images, following the CIFAR-10-C evaluation idea [4]. CIFAR-10-C is useful because it separates common corruptions into named corruption types and severity levels. It is not an adversarial benchmark. The corruptions are ordinary image degradations: noise, blur, weather effects, digital compression, contrast changes, and geometric distortions.

The project does not try to design a new backbone architecture. The backbone is ResNet18 [7], and the representation is trained with SimCLR, a self-supervised contrastive method [1]. The main question is narrower: after the model has been trained on source data, can it adapt itself at test time using only an unlabeled corrupted batch? If this adaptation improves accuracy, the model becomes less dependent on perfectly clean test data. If it decreases accuracy, the adaptation objective is not aligned well enough with classification.

The adaptation strategy implemented in the project is ActMAD-style activation matching [5]. The model first records activation statistics on clean source data. At test time, it receives a corrupted batch and updates a limited subset of parameters so that the target activations become closer to the source activations. The current implementation adapts normalization parameters and keeps the rest of the model frozen. The classification head is then applied after this short adaptation.

In the experiment we used 100 epochs of SimCLR pretraining. Linear evaluation reached 67.05%, and the selected test-time evaluation improved from 48.34% without adaptation to 55.78% after ActMAD TTT. The gain is 7.44 percentage points. A longer run, for example 200 epochs or more, would likely improve the representation, but it was not practical on the available computers because it would need faster memory and more compute.

2 Scientific Background and Related Work

2.1 Supervised image classification and distribution shift

In supervised image classification, the model learns a function

$$f_\theta : \mathcal{X} \to \{1, \dots, K\}$$

from images to class labels. For CIFAR-10, $K = 10$. The usual training objective assumes that the training and test examples are sampled from similar distributions. Under that condition, minimizing the training loss can lead to good test accuracy if the model generalizes correctly.

The problem changes when the test distribution is shifted. A corrupted image may still contain the same semantic object, but the low-level statistics can be different. Noise modifies pixel values. Blur removes high-frequency details. Compression creates blocks and artifacts. Fog and brightness changes affect contrast and color distributions. Since convolutional neural networks learn a mixture of low-level and high-level features, these changes can propagate through the network and modify the internal activations.

CIFAR-10-C was introduced as part of the common corruptions benchmark proposed by Hendrycks and Dietterich [4]. The benchmark is useful because it avoids vague robustness claims. A method can be tested on fixed corruptions and severity levels. If the full benchmark is used, accuracy or error can be reported per corruption and then averaged.

2.2 Self-supervised representation learning

Self-supervised learning uses labels created from the data itself. The goal is not to predict the final class label directly, but to learn a representation that contains useful structure. After pretraining, a smaller supervised head can be trained on top of the frozen representation. This is often called linear evaluation when the head is a linear classifier.

SimCLR is a contrastive self-supervised method [1]. For each image, two different augmented views are generated. The model should map these two views close to each other in representation space. Views from different images should be separated. This gives a training signal without class labels. For this project, SimCLR is a good fit because CIFAR-10 can provide many augmented image pairs, and the learned backbone can later be reused for classification and test-time adaptation.

The implementation uses a ResNet18 backbone and a projection head. The backbone produces a feature vector. The projection head maps this feature vector into the contrastive space used for the SimCLR loss. The classifier is not trained during contrastive pretraining. It is trained later during linear evaluation on frozen features.

2.3 Contrastive learning and NT-Xent loss

Let x_i be an image and let t_1 and t_2 be two stochastic augmentations. SimCLR builds two views:

$$\tilde{x}_{i,1} = t_1(x_i), \qquad \tilde{x}_{i,2} = t_2(x_i).$$

The encoder and projection head map them to normalized vectors $z_{i,1}$ and $z_{i,2}$. These two vectors are a positive pair. The other views in the same batch act as negative examples.

The project uses the NT-Xent loss. For a positive pair (i, j), a simplified form is:

$$\ell_{i,j} = -\log \frac{\exp(\mathrm{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\mathrm{sim}(z_i, z_k)/\tau)},$$

where sim is a similarity score and τ is the temperature. In the code, the projected vectors are normalized and the similarity matrix is computed by a dot product. The diagonal is masked because each view should not be compared with itself. The target label for each view is the index of its paired augmented view.

This loss is not a classification loss. It does not know the CIFAR-10 class names. It only asks the network to learn invariances induced by the augmentation pipeline. If the augmentations are well chosen, the representation captures useful image structure. If the augmentations are too weak or too strong, the representation can become less useful for classification.

2.4 Linear evaluation

Linear evaluation is a practical check of representation quality. After SimCLR pretraining, the backbone is frozen. A linear classifier is trained on top of the features using class labels. A high linear evaluation accuracy means that the frozen representation already separates classes well enough for a simple head. A low linear evaluation accuracy means that the representation is weak or not aligned with the classification task.

This matters for TTT. Test-time adaptation cannot fully repair a poor representation. If the backbone does not extract useful features before adaptation, then matching activation statistics may not be enough. The recorded 100-epoch run reached 67.05% linear evaluation accuracy. This is much stronger than the earlier one-epoch smoke result, but it is still limited compared with a strong fully supervised CIFAR-10 classifier. The number is therefore useful evidence of learned features, not evidence that the backbone is optimal.

2.5 Test-Time Training and test-time adaptation

Test-Time Training adapts a model during inference using an auxiliary objective that does not need labels [2]. A model trained on a source domain receives target-domain inputs at test time. Before predicting labels, it performs a few gradient steps on a self-supervised or unsupervised loss. The goal is to move the model toward the target distribution without using target labels.

The difficulty is that the auxiliary loss is not the classification loss. It may improve the internal representation, but it may also move the model in a harmful direction. TTT++ studies this instability and shows that self-supervised test-time training can fail under some shifts [3]. A positive result on one evaluated setting is useful, but it does not prove that the method is safe for all corruptions.

Test-time adaptation is attractive because it does not require labeled target data. In many applications, labeled target data is not available when the distribution changes. The model only sees new unlabeled images. This matches the project setting: target batches from corrupted CIFAR-10 images are available, but their labels should not be used for adaptation.

2.6 ActMAD and activation matching

ActMAD proposes to adapt a model by matching activation distributions between source data and test-time data [5]. The intuition is that a distribution shift changes internal activations. If the model can adjust itself so that target activations become closer to source activations, its predictions may become more stable.

The project implements a simplified ActMAD-style objective. During a source-statistics phase, hooks record the activations of normalization layers on clean data. For each layer, the code computes

a mean and a standard deviation. During adaptation, the target batch is passed through the model, target activation statistics are computed, and the loss penalizes the difference between target and source statistics. In simplified notation, for layers $l \in \mathcal{L}$:

$$\mathcal{L}_{\text{ActMAD}} = \frac{1}{|\mathcal{L}|} \sum_{l \in \mathcal{L}} \left(\|\mu_l^t - \mu_l^s\|_2^2 + \|\sigma_l^t - \sigma_l^s\|_2^2 \right).$$

Here, μ_l^s and σ_l^s are source statistics. The target statistics μ_l^t and σ_l^t are computed on the current unlabeled target batch. This loss does not use image labels.

3 Problem Formulation

The source domain is the clean CIFAR-10 distribution. It contains image-label pairs (x_s, y_s) sampled from D_s. The target domain contains corrupted CIFAR-10 images sampled from D_t. The labels still correspond to the same ten CIFAR-10 classes, but they are not used during adaptation.

The baseline prediction is obtained by applying the trained backbone and classifier directly to the corrupted batch:

$$\hat{y}_{\text{base}} = \arg\max f_\theta(x_t).$$

The adapted prediction first creates a temporary adapted model. A few gradient steps are applied on the unlabeled target batch using the activation matching loss. The prediction is then computed with the adapted parameters:

$$\hat{y}_{\text{ttt}} = \arg\max f_{\theta'}(x_t), \qquad \theta' = \theta - \alpha \nabla_\theta \mathcal{L}_{\text{ActMAD}}(x_t).$$

In the current implementation, not all parameters are adapted. Most of the model is frozen, and only normalization layer parameters are updated.

The evaluation compares top-1 accuracy before and after adaptation. The improvement is reported in percentage points:

$$\Delta = \text{Acc}_{\text{TTT}} - \text{Acc}_{\text{base}}.$$

A positive value means the adaptation improved accuracy. A negative value means the adaptation damaged accuracy. This sign is important because test-time adaptation should not be described as successful if it improves one setting and fails on another.

We have three cases: The first is clean CIFAR-10 source training and linear evaluation. The second is the selected test-time evaluation on corrupted CIFAR-10 data. The third is the larger all-corruption CIFAR-10-C evaluation.

4 Methodology

4.1 SimCLR pretraining on CIFAR-10

The first stage trains a self-supervised encoder on CIFAR-10. The loader returns two augmented views of each image. The augmentations used in the repository include random resized cropping, horizontal flipping, color jitter, random grayscale conversion, conversion to tensor, and CIFAR-

10 normalization. These transformations are defined in `src/datasets.py` through the two-view transformation used for SimCLR.

The model is defined in `src/self_supervised.py`. It uses a torchvision ResNet18 with the final classification layer replaced by an identity mapping. The output of the backbone is then passed through a projection head. The projection head is a small multilayer perceptron with a linear layer, ReLU activation, and another linear layer. The output is normalized before the contrastive loss.

For each batch, the training loop computes two projected representations, one for each view. The NT-Xent loss is then applied. The optimizer is Adam, with a cosine annealing scheduler.

4.2 Feature extraction and projection head

The backbone and projection head serve different roles. The projection head is useful for contrastive pretraining, but the classifier is trained on the raw backbone features. This is standard in SimCLR-style evaluation: the projection space helps the contrastive loss, while the representation before projection is used for downstream tasks.

In the code, the SimCLR forward path returns projected features for the SSL loss. The encoding path returns the raw backbone features. Linear evaluation uses the encoded backbone features, not the projection output. This distinction avoids forcing the final classifier to operate in the contrastive projection space.

4.3 Linear classifier on frozen features

After pretraining, the backbone parameters are frozen. The linear evaluation step trains a linear layer with ten output classes. In the inspected implementation, this classifier is trained with Adam. The classifier is saved as:

```
results/self_supervised/classifier.pt
```

The backbone checkpoint used for TTT is saved as:

```
results/self_supervised/simclr_backbone.pt
```

Linear evaluation is not the final project goal, but it gives an important diagnostic. The final value is 67.05%, which shows that the representation contains useful class information but is not yet very strong. The TTT result should be read with that limitation in mind.

4.4 Source activation statistics

Before test-time adaptation, the project computes source activation statistics. The code registers forward hooks on BatchNorm and LayerNorm modules. For each selected layer, it records activations on clean source validation data and computes mean and standard deviation across the appropriate dimensions.

The result is saved as:

```
results/self_supervised/source_stats.pt
```

This file is required by the TTT stage. It represents the clean source-domain activation reference. During adaptation, the corrupted target batch is pushed toward these reference statistics. Source statistics are computed on the backbone, while TTT wraps the backbone and classifier in a sequential model. The implementation therefore matches layer names after removing wrapper prefixes such as `0.`. If no source layer matches a target activation, adaptation stops instead of optimizing an empty ActMAD loss.

4.5 ActMAD-style adaptation at test time

The TTT stage is implemented in `src/test_time_training.py`. The adapter stores the base model and source statistics. For each target batch, the adapter creates a deep copy of the base model. This is important because test-time updates should not permanently modify the original model across unrelated batches. Without a fresh copy, adaptation errors could accumulate from batch to batch.

The copied model is put in training mode. All parameters are frozen first. Then, only parameters inside normalization modules are made trainable. The adaptation optimizer is Adam. The public TTT configuration uses ActMAD, ten adaptation steps per batch, learning rate 10^{-4}, and the source statistics stored in `results/self_supervised/source_stats.pt`.

At each adaptation step, the model processes the unlabeled target batch, the activation matching loss is computed, and the normalization parameters are updated. After the adaptation steps, the copied model is used for prediction. This design is cautious. It does not fine-tune the full network at test time. Updating all weights on a small unlabeled batch would be risky and expensive. Updating only normalization parameters is more limited, but it is less likely to destroy the learned representation.

5 Implementation Details

5.1 Implementation and repository structure

The repository is organized around two stages: Self-Supervised Learning (SSL) and Test-Time Training (TTT). The core code is in `src/`, experiment settings are in `configs/`, evaluation scripts are in `scripts/`, and datasets are expected under `data/`.

The single entry point is `run.py`. It accepts two main tasks:

- `self_supervised` or `ssl` for the pretraining phase;
- `test_time_training` or `ttt` for the adaptation phase.

Main implementation files:

- `run.py`: Handles command-line argument parsing, selects which task to run, loads model checkpoints, and manages the overall execution flow.
- `src/self_supervised.py`: Contains the entire SSL pipeline. This includes the SimCLR architecture, the projection head, the NT-Xent loss, the training loop, and the linear evaluation step. It is also responsible for extracting the source-domain statistics.

- `src/test_time_training.py`: Implements the test-time adaptation logic. This covers the activation hooks needed to gather statistics, the ActMAD loss function, and the TTT adapter itself.
- `src/datasets.py`: Manages data loading for standard CIFAR-10, SimCLR two-view augmentations, and corrupted CIFAR-10-C images.
- `src/metrics.py`: Computes top-1 accuracy for baseline and TTT evaluation.
- `scripts/eval_corruption.py`: Runs evaluation across the CIFAR-10-C corruption types.

5.2 Configuration files

Self-supervised configuration is `configs/self_supervised.yaml`. It uses CIFAR-10 at `data/raw`, enables download, and sets the training method to SimCLR.

Table 1: Self-supervised setting used for the reported SSL run.

Parameter	Value
Task	`self_supervised`
Configuration file	`configs/self_supervised.yaml`
Device in final log	`mps`
Dataset	CIFAR-10
Dataset root	`data/raw`
Backbone	ResNet18
Projection dimension	128
Method	SimCLR
Reported epochs	100
Batch size	256
Learning rate	0.03
Backbone output	`results/self_supervised/simclr_backbone.pt`
Classifier output	`results/self_supervised/classifier.pt`
Source statistics output	`results/self_supervised/source_stats.pt`

The TTT configuration is `configs/test_time_training.yaml`. It loads the saved SimCLR backbone, classifier, and source statistics from explicit paths. It uses `shot_noise` at severity 5, batch size 16, ten adaptation steps per batch, and learning rate 10^{-4}.

Table 2: Test-time adaptation configuration reported for `configs/test_time_training.yaml`.

Parameter	Value
Task	`test_time_training`
Alias	`ttt`
Configuration file	`configs/test_time_training.yaml`
Dataset	CIFAR-10-C style evaluation
Dataset root in config	`data/raw/cifar10c`
Resolved CIFAR-10-C directory	`data/raw/cifar10c/CIFAR-10-C/`
Corruption in config	`shot_noise`
Severity	5
Batch size	16
Classifier path	`results/self_supervised/classifier.pt`
Adaptation method	ActMAD
Steps per batch	10
Adaptation learning rate	0.0001

5.3 Checkpoints and result files

The terminal output confirms the following files after self-supervised training:

```
results/self_supervised/classifier.pt
results/self_supervised/simclr_backbone.pt
results/self_supervised/source_stats.pt
```

These files are produced by the self-supervised stage. The TTT stage loads them.

5.4 CIFAR-10-C data path

The TTT configuration sets the dataset root to:

```
data/raw/cifar10c
```

The loader then resolves the official CIFAR-10-C files under:

```
data/raw/cifar10c/CIFAR-10-C/
```

For the corruption `shot_noise`, the expected file is:

```
data/raw/cifar10c/CIFAR-10-C/shot_noise.npy
```

The labels are expected in:

```
data/raw/cifar10c/CIFAR-10-C/labels.npy
```

Each official corruption file contains 50,000 images, with 10,000 images per severity level. The loader selects the slice corresponding to the requested severity. For severity 5, it selects the last 10,000 images.

The final successful TTT command used the same `configs/test_time_training.yaml` path.

6 Experimental Protocol

6.1 Source-domain pretraining

The source-domain pretraining command was:

```
python run.py --task self_supervised --config configs/self_supervised.yaml
```

The alias is:

```
python run.py --task ssl --config configs/self_supervised.yaml
```

The loss was printed once per epoch. The initial loss was 5.6214 and the final recorded loss at epoch 100 was 4.6586.

6.2 Linear evaluation

After SimCLR pretraining, `run.py` calls the linear evaluation step. The backbone is frozen and a linear classifier is trained on top of the learned features. The classifier is then evaluated on CIFAR-10. The final linear evaluation accuracy was 67.05%.

The result is stronger than the earlier smoke run, but it still leaves space for improvement. A stronger CIFAR-10 encoder would normally be expected to improve both clean accuracy and robustness.

6.3 Test-time adaptation setup

The TTT command was:

```
python run.py --task test_time_training --config configs/test_time_training.yaml
```

The alias is:

```
python run.py --task ttt --config configs/test_time_training.yaml
```

The baseline accuracy was 48.34%, the ActMAD TTT accuracy was 55.78%, and the improvement was 7.44 percentage points.

6.4 All-corruption evaluation

The repository includes an all-corruption script:

```
python scripts/eval_corruption.py
```

the report material lists the 15 CIFAR-10-C corruptions:

```
gaussian_noise, shot_noise, impulse_noise, defocus_blur,
glass_blur, motion_blur, zoom_blur, snow, frost, fog,
brightness, contrast, elastic_transform, pixelate,
jpeg_compression
```

This script is the entry point for a larger run over the listed corruptions.

7 Experimental Results

7.1 Available numerical results

The available numbers are summarized in Table 3.

Table 3: Available numerical results.

Experiment	Value
Self-supervised run epochs	100
Initial SimCLR loss	5.6214
Final SimCLR loss	4.6586
Linear evaluation accuracy	67.05%
Baseline accuracy	48.34%
ActMAD TTT accuracy	55.78%
Improvement	+7.44 percentage points

Table 4: Selected TTT evaluation.

Evaluation setting	Baseline accuracy	ActMAD TTT accuracy	Improvement
Selected TTT evaluation	48.34%	55.78%	+7.44 pp

7.2 Training loss evolution

The saved SimCLR loss log decreased from 5.6214 at epoch 1 to 4.6586 at epoch 100. The curve in Figure 1 is built from the 100 losses printed in the terminal log.

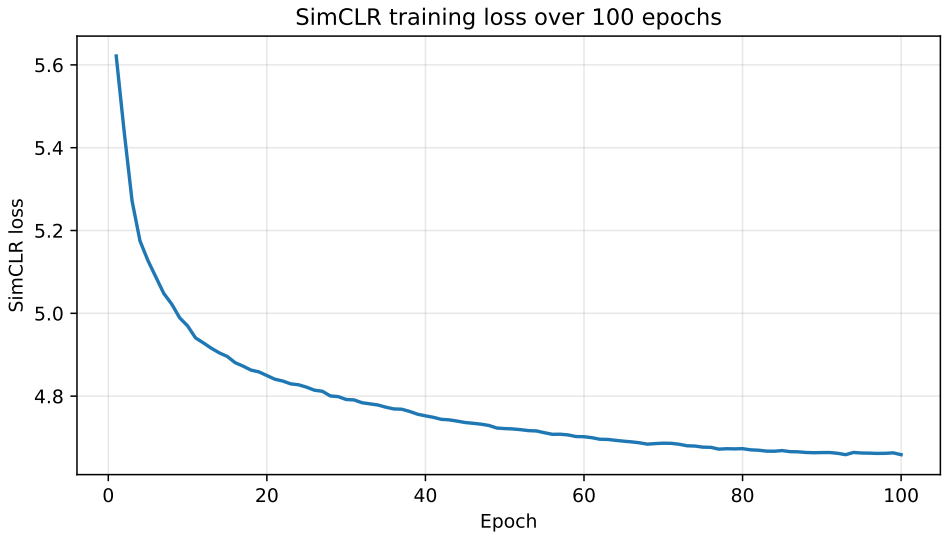

Figure 1: SimCLR training loss over the recorded 100-epoch run.

The decrease is strongest during the first epochs. After roughly epoch 70, the loss changes more

slowly and oscillates around a lower value. This does not prove that the representation is optimal, but it shows that the contrastive objective was optimized across the reported run.

Table 5: Selected SimCLR loss values from the final run.

Epoch	1	10	25	50	75	90	100
Loss	5.6214	4.9697	4.8217	4.7216	4.6768	4.6637	4.6586

8 Analysis and Discussion

The final selected evaluation gives 48.34% accuracy without adaptation and 55.78% after ActMAD TTT. The gain is 7.44 percentage points. This result supports the idea that activation matching can help under the evaluated shift.

The baseline accuracy under the selected shifted evaluation is lower than the 67.05% linear evaluation accuracy. This is consistent with the distribution-shift problem. The classifier works better on the clean evaluation setting than on the shifted test setting. The drop does not come from a new class set, because the CIFAR-10 classes remain the same. It comes from the change in image statistics seen by the backbone and classifier.

The ActMAD result recovers part of the lost performance. Matching target activation statistics to source activation statistics can move the adapted model toward the distribution on which the classifier was trained. In this run, that movement improved accuracy rather than damaging it. The improvement is large enough to be meaningful here: +7.44 percentage points is not just a rounding effect.

Test-time adaptation is not automatically safe. The model adapts without labels. A lower activation matching loss does not always mean better classification. If the target batch is small, unbalanced, or too corrupted, the statistics can be misleading. The optimizer may make the activations look more source-like while losing useful class information. This is why future experiments should keep negative results visible and not only report successful cases.

9 Limitations

The main limitation is evaluation coverage. The final TTT result is for one selected setting. The final files do not include `results/test_time_training/results.csv`, a table over all 15 CIFAR-10-C corruptions, or a mean score over all corruptions. The terminal output is enough to report the baseline and ActMAD accuracies for the selected run, but a saved CSV would make the result easier to check and repeat.

The settings were not fully compared. The report uses ten ActMAD steps and learning rate 10^{-4}. It does not compare several step counts, batch sizes, learning rates, or sets of adapted parameters.

ActMAD aligns activation statistics, not labels. A lower activation matching loss does not always mean better classification. This should be checked when reading both positive and negative results.

10 Reproducibility

The main commands from the repository root are:

```
python run.py --task self_supervised --config configs/self_supervised.yaml
python run.py --task test_time_training --config configs/test_time_training.yaml
python run.py --task ssl --config configs/self_supervised.yaml
python run.py --task ttt --config configs/test_time_training.yaml
python scripts/eval_corruption.py
```

The expected order is to run self-supervised training first. This creates the backbone, classifier, and source activation statistics under `results/self_supervised/`. The TTT stage then loads these files. The confirmed saved files are:

```
results/self_supervised/simclr_backbone.pt
results/self_supervised/classifier.pt
results/self_supervised/source_stats.pt
```

The local CSV output path is:

```
results/test_time_training/results.csv
```

For full CIFAR-10-C evaluation, the official `.npy` files and `labels.npy` must exist under:

```
data/raw/cifar10c/CIFAR-10-C/
```

The all-corruption script can be used for a wider evaluation.

The terminal warnings observed during the provided run are not central to the scientific method. The MPS warning says that `pin_memory=True` is not supported on MPS, so pinned memory is not used. This can be cleaned by setting `pin_memory=False` when the device is MPS. The NumPy deprecation warning comes from dataset loading and does not change the reported accuracy values, but a stable dependency combination is safer for repeated experiments.

11 Final Remarks and Future Work

The completed 100-epoch run shows that the SSL backbone is usable, but still limited. In the selected corrupted setting, ActMAD improves accuracy by 7.44 percentage points, from 48.34% to 55.78%.

The next practical step is to run `scripts/eval_corruption.py`, save the resulting CSV, and compare ActMAD across all listed corruptions and severity levels. It would also be useful to test several adaptation step counts, several batch sizes, and at least one other test-time adaptation method.

12 References

References

[1] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on*

Machine Learning, 2020.

[2] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros, and M. Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

[3] Y. Liu, P. Kothari, B. G. van Delft, B. Bellot-Gurlet, T. Mordan, and A. Alahi. TTT++: When Does Self-Supervised Test-Time Training Fail or Thrive? In *Advances in Neural Information Processing Systems*, 2021.

[4] D. Hendrycks and T. Dietterich. Benchmarking Neural Network Robustness to Common Corruptions and Perturbations. In *International Conference on Learning Representations*, 2019.

[5] M. J. Mirza, P. Jané Soneira, W. Lin, M. Kozinski, H. Possegger, and H. Bischof. ActMAD: Activation Matching to Align Distributions for Test-Time-Training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

[6] A. Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto, 2009.

[7] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.